

Formalizing Rust's Type System

Jacques-Henri Jourdan

February 18th, 2026

Agenda for next weeks

- Next week 2026/02/25: **no course**
- 2026/03/03: deadline for programming project
- 2026/03/04: course here with me
- 2026/03/11: final exam

Abstract from last weeks

During the last three weeks, we had an introduction to Rust programming

- Type system enforces: “mutation XOR aliasing”,
- Traits: an abstraction mechanism, sometimes at zero-cost.
- Unsafe blocks/functions: workaround strong static type-checking constraints,
- Encapsulation: clients can safely use libraries written with unsafe code.
- Interior mutability (the ability to mutate through shared borrows): a typical example of well-encapsulated unsafe code.
- Rust is a good fit for multithreading.
 - Protects against data races (with `Send` and `Sync`).
 - Provides abstractions for multithreading.

I keep telling you Rust is type-safe.
How can I be so sure?

I.e., is this something one can prove formally?

1 Introduction

2 Syntactic type system

- λ_{Rust}
- Type system

3 Building the logical relation

- Setup
- Interpreting simple types
- Lifetime logic

4 Supporting unsafe code

Introduction

Syntactic type
system

λ_{Rust}

Type system

Building the logical
relation

Setup

Interpreting simple
types

Lifetime logic

Supporting unsafe
code

The problem we are trying to solve

“Well-typed Rust programs do not go wrong.”

Really?

Introduction

Syntactic type
system

λ_{Rust}

Type system

Building the logical
relation

Setup

Interpreting simple
types

Lifetime logic

Supporting unsafe
code

The problem we are trying to solve

Introduction

Syntactic type
system

λ_{Rust}

Type system

Building the logical
relation

Setup

Interpreting simple
types

Lifetime logic

Supporting unsafe
code

“Well-typed Rust programs **not using** `unsafe` do not go wrong.”

Is that all?

The problem we are trying to solve

Introduction

Syntactic type
system

λ_{Rust}
Type system

Building the logical
relation

Setup
Interpreting simple
types
Lifetime logic

Supporting unsafe
code

“Well-typed Rust programs **not using** `unsafe` do not go wrong.”

A vast majority of real-life Rust programs (directly or indirectly) use unsafe code.
Example: `Vec`, interior mutability (e.g., `Cell`), low-level optimizations, ...

We want an **extensible** theorem to prove those programs safe too.

- Safe pieces of code are safe thanks to **syntactic typing rules**.
- Unsafe pieces are safe thanks to a **specialized proof**.

Both kinds of proofs will be linked together thanks to a **logical relation**.

Logical relations for type safety

The general approach

A type system is defined from:

- One (or several) **typing judgment(s)** $\dots \vdash \dots$
- Syntactic **typing rules**.

Ex. for lambda-calculus:
$$\frac{\Gamma \vdash e_1 : \tau_1 \rightarrow \tau_2 \quad \Gamma \vdash e_2 : \tau_1}{\Gamma \vdash e_1 e_2 : \tau_2}$$

The logical relation approach to type soundness (i.e., semantic type soundness):

- 1 Define a **semantic typing judgment** $\dots \models \dots$ from the operational semantics.
- 2 Prove the **fundamental theorem**: “ $\dots \vdash \dots \Rightarrow \dots \models \dots$ ”
- 3 Prove **adequacy** for the logical relation: “ $\dots \models \dots \Rightarrow \text{safety}$ ”.
 - “**Semantically** well-typed programs do not go wrong.”
 - Usually easy from the definition of $\dots \models \dots$.

Why is this approach more extensible than subject reduction+progress?

Extending semantic type safety

Introduction

Syntactic type
system

λ_{Rust}

Type system

Building the logical
relation

Setup

Interpreting simple
types

Lifetime logic

Supporting unsafe
code

The fundamental theorem: by induction over the typing tree using **semantic typing rules**:

$$\frac{\begin{array}{c} \dots \vdash \dots \\ \hline \dots \vdash \dots \\ \vdots \\ \dots \vdash \dots \end{array} \quad \dots \vdash \dots}{\dots \vdash \dots} \quad \dots \vdash \dots \Rightarrow \frac{\begin{array}{c} \dots \models \dots \\ \hline \dots \models \dots \\ \vdots \\ \dots \models \dots \end{array} \quad \dots \models \dots}{\dots \models \dots} \quad \dots \models \dots$$

Extending semantic type safety

The fundamental theorem: by induction over the typing tree using **semantic typing rules**:

$$\frac{\begin{array}{c} \dots \vdash \dots \\ \hline \dots \vdash \dots \\ \vdots \\ \dots \vdash \dots \end{array} \quad \dots \vdash \dots}{\dots \vdash \dots} \quad \dots \vdash \dots \Rightarrow \frac{\begin{array}{c} \dots \models \dots \\ \hline \dots \models \dots \\ \vdots \\ \dots \models \dots \end{array} \quad \dots \models \dots}{\dots \models \dots}$$

To prove $\dots \models \dots$ for the whole program, we don't need $\dots \vdash \dots$ everywhere.
We can fill some “holes” using custom proofs of $\dots \models \dots$.

Extending semantic type safety

Introduction

Syntactic type
system

λ_{Rust}
Type system

Building the logical
relation

Setup
Interpreting simple
types
Lifetime logic

Supporting unsafe
code

`Cell::get` is not syntactically well-typed in safe Rust: “ $\dots \not\vdash \text{Cell}::\text{get} : \dots$ ”.

But we can **prove** “ $\dots \models \text{Cell}::\text{get} : \dots$ ”.

Combining with an otherwise syntactically well-typed program:

$$\frac{\begin{array}{c} \dots \vdash \dots \\ \hline \dots \vdash \dots \\ \vdots \\ \dots \vdash \text{Cell}::\text{get} : \dots \end{array}}{\dots \models \dots} \quad \dots \vdash \dots$$
$$\frac{\dots \models \dots}{\dots \models \dots}$$

And we can conclude safety thanks to adequacy!

What's left to be done...

- Define formally the language and its syntactic type system.
- Define the logical relation and prove adequacy+fundamental theorem.
- Extend the logical relation for types like `Cell`...

That's a lot of work.

We will only see a [sketch here](#).

The details are in a paper, and the accompanying Rocq development:

RustBelt: Securing the foundations of the Rust programming language. In POPL'18.

To make sure nothing is wrong, this is [formalized with Rocq](#). Particularly [useful](#) to design and maintain the proof.

1 Introduction

2 Syntactic type system

- λ_{Rust}
- Type system

3 Building the logical relation

- Setup
- Interpreting simple types
- Lifetime logic

4 Supporting unsafe code

Introduction

**Syntactic type
system**

λ_{Rust}

Type system

Building the logical
relation

Setup

Interpreting simple
types

Lifetime logic

Supporting unsafe
code

The language: λ_{Rust}

Rust is **way too complex** to be formalized as-is.

Should we formalize MIR (recall: the language behind the borrow checker)?
It still has a lot of technicalities unrelated to the type system, refers to traits...

Instead, we formalize a **core language**.

It has most of the features of MIR relevant for type soundness, but **idealized** to make the theory simpler.

The language: λ_{Rust}

Paths, elementary values

$$\begin{aligned} \text{Val} \ni v &::= \text{false} \mid \text{true} \mid z \mid \ell \mid \text{funrec } f(\bar{x}) \text{ ret } k := F \\ \text{Path} \ni p &::= x \mid p.n \end{aligned}$$

Elementary values that can be stored in local registers $x, \dots$

Complex values (e.g., `struct`) are stored in memory, and accessed through pointers.

Note: function values take a continuation in parameter (language in CFG, see later).

The language: λ_{Rust}

Paths, elementary values

$$\text{Val} \ni v ::= \text{false} \mid \text{true} \mid z \mid \ell \mid \text{funrec } f(\bar{x}) \text{ ret } k := F$$
$$\text{Path} \ni p ::= x \mid p.n$$

Pointers to fields of complex values can be created with **paths**.

$p.n$: pointer to field at offset n of **struct** pointed to by p .

Operationally: just a pointer offset (i.e., an addition).

The language: λ_{Rust}

Instructions

$$\begin{aligned} \text{Instr} \ni I ::= & v \mid p \mid p_1 + p_2 \mid p_1 - p_2 \mid p_1 \leq p_2 \mid p_1 == p_2 \\ & \mid \text{new}(n) \mid \text{delete}(n, p) \mid *p \mid p_1 := p_2 \mid p_1 :=_n *p_2 \\ & \mid \dots \end{aligned}$$

Instructions are the elementary operations of λ_{Rust} .

They take **paths** as operands.

The language: λ_{Rust}

Instructions

$$\begin{aligned} \text{Instr} \ni I ::= & v \mid p \mid p_1 + p_2 \mid p_1 - p_2 \mid p_1 \leq p_2 \mid p_1 == p_2 \\ & \mid \text{new}(n) \mid \text{delete}(n, p) \mid *p \mid p_1 := p_2 \mid p_1 :=_n *p_2 \\ & \mid \dots \end{aligned}$$

λ_{Rust} 's instructions include values, paths, arithmetic operations...

The language: λ_{Rust}

Instructions

$$\begin{aligned} \text{Instr} \ni I ::= & v \mid p \mid p_1 + p_2 \mid p_1 - p_2 \mid p_1 \leq p_2 \mid p_1 == p_2 \\ & \mid \text{new}(n) \mid \text{delete}(n, p) \mid *p \mid p_1 := p_2 \mid p_1 :=_n *p_2 \\ & \mid \dots \end{aligned}$$

... and instruction to allocate/free memory, and read/write it.

Read/write one word from/to a register: $*p, p_1 := p_2$.

■ Operational semantics defined so that races $\Rightarrow$ undefined behavior.

Copy a complex value (several words) from one memory location to another: $p_1 :=_n *p_2$.

The language: λ_{Rust}

Function bodies

$$\begin{aligned} \text{FuncBody} \ni F ::= & \text{letcont } k(\bar{x}) := F_1 \text{ in } F_2 \mid \text{jump } k(\bar{x}) \\ & \mid \text{let } x = l \text{ in } F \mid \text{if } p \text{ then } F_1 \text{ else } F_2 \\ & \mid \text{call } f(\bar{x}) \text{ ret } k \\ & \mid \text{newlft}; F \mid \text{endlft}; F \\ & \mid \dots \end{aligned}$$

Function bodies combine instructions to build functions.

Code is written in CPS: a way to model code written as a CFG.

We can declare a new continuation (i.e., label in the CFG), and jump to it.

The language: λ_{Rust}

Function bodies

$$\begin{aligned} \text{FuncBody} \ni F ::= & \text{letcont } k(\bar{x}) := F_1 \text{ in } F_2 \mid \text{jump } k(\bar{x}) \\ & \mid \text{let } x = l \text{ in } F \mid \text{if } p \text{ then } F_1 \text{ else } F_2 \\ & \mid \text{call } f(\bar{x}) \text{ ret } k \\ & \mid \text{newlft}; F \mid \text{endlft}; F \\ & \mid \dots \end{aligned}$$

Basic function bodies:

- An instruction (+ binding the result to a variable).
- If-then-else.
- Calling a function (passing a continuation).

The language: λ_{Rust}

Function bodies

$$\begin{aligned} \text{FuncBody} \ni F ::= & \text{letcont } k(\bar{x}) := F_1 \text{ in } F_2 \mid \text{jump } k(\bar{x}) \\ & \mid \text{let } x = l \text{ in } F \mid \text{if } p \text{ then } F_1 \text{ else } F_2 \\ & \mid \text{call } f(\bar{x}) \text{ ret } k \\ & \mid \text{newlft}; F \mid \text{endlft}; F \\ & \mid \dots \end{aligned}$$

Ghost instructions for creating/ending a lifetime.

The language: λ_{Rust}

Handling sum types

$$\begin{aligned} \text{Instr} \ni I &::= \dots \\ &\quad | p \stackrel{\text{inj } i}{=} () \mid p_1 \stackrel{\text{inj } i}{=} p_2 \mid p_1 \stackrel{\text{inj } i}{=}_n * p_2 \\ &\quad | \dots \\ \text{FuncBody} \ni F &::= \dots \\ &\quad | \text{case } *p \text{ of } \bar{F} \end{aligned}$$

λ_{Rust} has a notion of sum types, stored in memory, with a tag in the first word.

There are special instructions to write such values together with the tag.

And a case statement for “pattern matching” the in-memory tag.

$$\begin{aligned} \text{Type} \ni \tau ::= & \text{bool} \mid \text{int} \mid \text{own } \tau \mid \&_{\text{mut}}^{\kappa} \tau \mid \&_{\text{shr}}^{\kappa} \tau \mid \downarrow_n \\ & \mid \Pi \bar{\tau} \mid \Sigma \bar{\tau} \mid \forall \bar{\alpha}. \text{fn}(\text{f} : \text{E}; \bar{\tau}) \rightarrow \tau \mid T \mid \mu T. \tau \\ & \mid \dots \end{aligned}$$

Types for integer and Booleans.

$$\begin{aligned} \text{Type} \ni \tau ::= & \text{bool} \mid \text{int} \mid \text{own } \tau \mid \&_{\text{mut}}^{\kappa} \tau \mid \&_{\text{shr}}^{\kappa} \tau \mid \downarrow_n \\ & \mid \Pi \bar{\tau} \mid \Sigma \bar{\tau} \mid \forall \bar{\alpha}. \text{fn}(\text{f} : \text{E}; \bar{\tau}) \rightarrow \tau \mid T \mid \mu T. \tau \\ & \mid \dots \end{aligned}$$

Types for pointers: `Box` is written `own  $\tau$` .

$$\begin{aligned} \text{Type} \ni \tau ::= & \mathbf{bool} \mid \mathbf{int} \mid \mathbf{own} \tau \mid \&_{\mathbf{mut}}^{\kappa} \tau \mid \&_{\mathbf{shr}}^{\kappa} \tau \mid \textcolor{brown}{\downarrow} n \\ & \mid \Pi \bar{\tau} \mid \Sigma \bar{\tau} \mid \forall \bar{\alpha}. \mathbf{fn}(\mathbf{f} : \mathbf{E}; \bar{\tau}) \rightarrow \tau \mid T \mid \mu T. \tau \\ & \mid \dots \end{aligned}$$

Type of “uninitialized memory”.

- When memory is just allocated, or when non-Copy values are moved.
- Used to replace the “initializedness” analysis of the borrow checker.

$$\begin{aligned} \text{Type} \ni \tau ::= & \text{bool} \mid \text{int} \mid \text{own } \tau \mid \&_{\text{mut}}^{\kappa} \tau \mid \&_{\text{shr}}^{\kappa} \tau \mid \downarrow_n \\ & \mid \Pi \bar{\tau} \mid \Sigma \bar{\tau} \mid \forall \bar{a}. \text{fn}(\text{f} : \text{E}; \bar{\tau}) \rightarrow \tau \mid T \mid \mu T. \tau \\ & \mid \dots \end{aligned}$$

Complex types: sum, products.

Used to model `struct`, `enum` and tuples.

$$\begin{aligned} \text{Type} \ni \tau ::= & \text{bool} \mid \text{int} \mid \text{own } \tau \mid \&_{\text{mut}}^{\kappa} \tau \mid \&_{\text{shr}}^{\kappa} \tau \mid \downarrow_n \\ & \mid \Pi \bar{\tau} \mid \Sigma \bar{\tau} \mid \forall \bar{\alpha}. \text{fn}(\text{f} : \text{E}; \bar{\tau}) \rightarrow \tau \mid T \mid \mu T. \tau \\ & \mid \dots \end{aligned}$$

Type of function pointers, guarded equirecursive types...

$$\begin{aligned} \text{Type} \ni \tau ::= & \text{bool} \mid \text{int} \mid \text{own } \tau \mid \&_{\text{mut}}^{\kappa} \tau \mid \&_{\text{shr}}^{\kappa} \tau \mid \downarrow_n \\ & \mid \Pi \bar{\tau} \mid \Sigma \bar{\tau} \mid \forall \bar{\alpha}. \text{fn}(\text{f} : \text{E}; \bar{\tau}) \rightarrow \tau \mid T \mid \mu T. \tau \\ & \mid \dots \end{aligned}$$

Extensible for types providing safe abstraction over **unsafe**!

Typing judgments

There are two main typing judgments:

- They model both the **type checker** and the **borrow checker** of the Rust compiler.
- One for instructions: $E; L \mid T_1 \vdash I \dashv x. T_2$
- One for function bodies: $E; L \mid K; T \vdash F$

They depend on **four different kinds of contexts**:

- T is the **typing context**. Elements:
 - $p \triangleleft \tau$: path p contains an elementary value of type τ .
 - $p \triangleleft^{\dagger \kappa} \tau$: same, but **frozen** until lifetime κ ends.
 - **Substructural**: elements cannot be duplicated (ownership tracking!).
- E and L are **lifetime contexts**. They contain:
 - information on **lifetime inclusion**, and
 - information on **local lifetimes**: which are they, and when they can be ended.
- K is the **continuation context**.
 - Describes the continuations we can jump to, and the required contexts.
 - Elements: $k \triangleleft \text{cont}(L; \bar{x}. T)$.
 "We can jump to k , passing parameters $\bar{x}$, if the contexts contain L and T ."

Typing judgments

There are two main typing judgments:

- They model both the **type checker** and the **borrow checker** of the Rust compiler.
- One for instructions: $E; L \mid T_1 \vdash I \dashv x. T_2$
- One for function bodies: $E; L \mid K; T \vdash F$

They depend on **four different kinds of contexts**:

- T is the **typing context**. Elements:
 - $p \triangleleft \tau$: path p contains an elementary value of type τ .
 - $p \triangleleft^{\uparrow \kappa} \tau$: same, but **frozen** until lifetime κ ends.
 - **Substructural**: elements cannot be duplicated (ownership tracking!).
- E and L are **lifetime contexts**. They contain:
 - information on **lifetime inclusion**, and
 - information on **local lifetimes**: which are they, and when they can be ended.
- K is the **continuation context**.
 - Describes the continuations we can jump to, and the required contexts.
 - Elements: $k \triangleleft \text{cont}(L; \bar{x}. T)$.
 “We can jump to k , passing parameters $\bar{x}$, if the contexts contain L and T .”

Typing judgments

There are two main typing judgments:

- They model both the **type checker** and the **borrow checker** of the Rust compiler.
- One for instructions: $E; L \mid T_1 \vdash I \dashv x. T_2$
- One for function bodies: $E; L \mid K; T \vdash F$

They depend on **four different kinds of contexts**:

- T is the **typing context**. Elements:
 - $p \triangleleft \tau$: path p contains an elementary value of type τ .
 - $p \triangleleft^{\uparrow \kappa} \tau$: same, but **frozen** until lifetime κ ends.
 - **Substructural**: elements cannot be duplicated (ownership tracking!).
- E and L are **lifetime contexts**. They contain:
 - information on **lifetime inclusion**, and
 - information on **local lifetimes**: which are they, and when they can be ended.
- K is the **continuation context**.
 - Describes the continuations we can jump to, and the required contexts.
 - Elements: $k \triangleleft \mathbf{cont}(L; \bar{x}. T)$.
 “We can jump to k , passing parameters $\bar{x}$, if the contexts contain L and T .”

Typing judgments

There are two main typing judgments:

- They model both the **type checker** and the **borrow checker** of the Rust compiler.
- One for instructions: $E; L \mid T_1 \vdash I \dashv x. T_2$
- One for function bodies: $E; L \mid K; T \vdash F$

The typing context for instructions has two typing contexts: **an input typing context T_1** and **an output typing context T_2** .

The instruction I **consumes T_1** and **produces T_2** .

$x.T_2$ binds a special variable x : to give a type to the result of the instruction.

Function bodies never return (CPS). They only consume T .

Typing rules

Typing instructions

Selected rules:

$$E; L \mid \emptyset \vdash z \dashv x. x \triangleleft \mathbf{int}$$

$$E; L \mid p_1 \triangleleft \mathbf{int}, p_2 \triangleleft \mathbf{int} \vdash p_1 \leq p_2 \dashv x. x \triangleleft \mathbf{bool}$$

$$E; L \mid \emptyset \vdash \mathbf{new}(n) \dashv x. x \triangleleft \mathbf{own} \not\triangleleft n$$

$$\frac{n = \mathbf{size}(\tau)}{E; L \mid p \triangleleft \mathbf{own} \tau \vdash \mathbf{delete}(n, p) \dashv \emptyset}$$

$$\frac{\tau \text{ copy} \quad \mathbf{size}(\tau) = 1}{E; L \mid p \triangleleft \mathbf{own} \tau \vdash *p \dashv x. p \triangleleft \mathbf{own} \tau, x \triangleleft \tau}$$

$$\frac{\mathbf{size}(\tau) = 1}{E; L \mid p \triangleleft \mathbf{own} \tau \vdash *p \dashv x. p \triangleleft \mathbf{own} \not\triangleleft 1, x \triangleleft \tau}$$

$$\frac{E; L \vdash \kappa \text{ alive}}{E; L \mid p_1 \triangleleft \&_{\mathbf{mut}}^{\kappa} \tau, p_2 \triangleleft \tau \vdash p_1 := p_2 \dashv p_1 \triangleleft \&_{\mathbf{mut}}^{\kappa} \tau}$$

Typing rules

Typing function bodies

Selected rules:

$$\frac{E; L \mid T_1 \vdash I \dashv x. T_2 \quad E; L \mid K; T_2, T \vdash F}{E; L \mid K; T_1, T \vdash \text{let } x = I \text{ in } F}$$

$$\frac{k \triangleleft \mathbf{cont}(L; \bar{x}. T) \in K}{E; L \mid K; T[\bar{y}/\bar{x}] \vdash \text{jump } k(\bar{y})}$$

$$\frac{E; L \vdash T \rightleftharpoons T' \quad E; L \mid K; T' \vdash F}{E; L \mid K; T \vdash F}$$

Helper judgment for transforming a typing context: $E; L \vdash T \rightleftharpoons T'$.

Typing rules

Transforming typing environments

Some transformations can happen on environments before typing a piece of code:

$$\frac{\tau \text{ copy}}{E; L \vdash p \triangleleft \tau \xrightarrow{\text{ctx}} p \triangleleft \tau, p \triangleleft \tau}$$

$$E; L \vdash p \triangleleft \&_{\mu}^{\kappa} (\tau_1 \times \tau_2) \xrightarrow{\text{ctx}} p.0 \triangleleft \&_{\mu}^{\kappa} \tau_1, p.\text{size}(\tau_1) \triangleleft \&_{\mu}^{\kappa} \tau_2$$

$$\frac{E; L \vdash \kappa \text{ alive}}{E; L \vdash p \triangleleft \&_{\text{mut}}^{\kappa} \tau \xrightarrow{\text{ctx}} p \triangleleft \&_{\text{shr}}^{\kappa} \tau}$$

$$E; L \vdash p \triangleleft \text{own}_n \tau \xrightarrow{\text{ctx}} p \triangleleft \&_{\text{mut}}^{\kappa} \tau, p \triangleleft^{\dagger \kappa} \text{own}_n \tau$$

$$\frac{E; L \vdash \kappa' \sqsubseteq \kappa}{E; L \vdash p \triangleleft \&_{\text{mut}}^{\kappa} \tau \xrightarrow{\text{ctx}} p \triangleleft \&_{\text{mut}}^{\kappa'} \tau, p \triangleleft^{\dagger \kappa'} \&_{\text{mut}}^{\kappa} \tau}$$

1 Introduction

2 Syntactic type system

- λ_{Rust}
- Type system

3 Building the logical relation

- Setup
- Interpreting simple types
- Lifetime logic

4 Supporting unsafe code

Recap

We have presented **syntactic typing judgments** $E; L \mid T_1 \vdash I \dashv x. T_2$ and $E; L \mid K; T \vdash F$.

We need to:

- Give them **semantic counterparts** $E; L \mid T_1 \models I \dashv x. T_2$ and $E; L \mid K; T \models F$
- Prove adequacy: if $\emptyset \mid \emptyset; \emptyset \mid \emptyset \models f \dashv x. x \triangleleft \mathbf{fn}() \rightarrow \Pi[]$, then f cannot execute to a stuck state.
- Prove the **semantic typing rules**: same as syntactic rules, but replacing $\vdash$ with $\models$.
- Extend the proof to types defined using **unsafe**.

The main difficulty is to find the definitions of the semantic judgments.

Once defined, their properties follow “naturally” (somewhat). In this course, we will **not give the proofs** and focus on the definitions.

Let's define a model for semantic judgments.

To do that, we need a semantic interpretation of types: $\llbracket \tau \rrbracket$.

Problem: we need to speak about **ownership**. Any ideas how to model this?

Separation Logic

to the Rescue!



Formalizing Rust's
Type System

Jacques-Henri
Jourdan

Introduction

Syntactic type
system

λ_{Rust}

Type system

Building the logical
relation

Setup

Interpreting simple
types

Lifetime logic

Supporting unsafe
code

Separation Logic



to the Rescue!

Extension of Hoare logic (O'Hearn-Reynolds-..., 1999)

- For reasoning about pointer-manipulating programs

Major influence on many verification & analysis tools

- e.g. Infer, VeriFast, Viper, Bedrock, jStar, ...

Separation logic = Ownership logic

- Perfect fit for modeling Rust's ownership types!

We use **Iris**, a modern and expressive separation logic.

It features:

- standard separation logic mechanisms,
- support for concurrency,
- built-in support for step-indexing (equirecursive types, higher-order store, ...),
- a powerful mechanism to create new mechanisms of ownership (borrows, ...).

Semantic interpretation of types

We define an **ownership predicate** $\llbracket \tau \rrbracket.\text{own}(t, \bar{v})$ in separation logic for every type τ .

- It represents what it **means** in separation logic that $\bar{v}$ is an object of type τ in thread t .
- It not only states what $\bar{v}$ should look like, it also describes the **resources** behind τ .

Note: it depends on the thread t : the interpretation of some types depend on it!
Why is this useful?

We define an **ownership predicate** $\llbracket \tau \rrbracket.\text{own}(t, \bar{v})$ in separation logic for every type τ .

- It represents what it **means** in separation logic that $\bar{v}$ is an object of type τ in thread t .
- It not only states what $\bar{v}$ should look like, it also describes the **resources** behind τ .

Note: it depends on the thread t : the interpretation of some types depend on it!

Why is this useful?

Semantic interpretation of **Send**: types for which $\llbracket \tau \rrbracket.\text{own}(t, \bar{v})$ do **not** depend on t .

$\Rightarrow$ The interpretation will still be valid in another thread.

Interpreting semantic environments

We directly lift semantic types to semantic environments using the separating conjunction:

$$\begin{aligned}\llbracket p \triangleleft \tau, T \rrbracket &:= \llbracket \tau \rrbracket.\text{own}(t, p) * \llbracket T \rrbracket \\ \llbracket \emptyset \rrbracket &:= \text{True}\end{aligned}$$

Meaning: every element in the typing context correspond to a distinct piece of ownership.

Semantic typing judgements

$$E; L \mid T_1 \models I \Rightarrow x. T_2 := \\ \forall t. \{ \llbracket E \rrbracket * \llbracket L \rrbracket * [Na : t] * \llbracket T_1 \rrbracket(t) \} / \{ v. \llbracket L \rrbracket * [Na : t] * \llbracket T_2 \rrbracket_{[x \leftarrow v]}(t) \}$$

Semantic typing judgment for function bodies is similar.

(I have omitted the definition of $\llbracket E \rrbracket$ and $\llbracket L \rrbracket$ – not important for understanding.)

We use a separation logic **Hoare triple**.

- All we say is that I behaves well when placed in an environment described by the contexts, and return an environment described by the output contexts.

$[Na : t]$ is a special resource owned by one thread only.

Non-thread-safe types need $[Na : t]$. Thus their interpretation depends on t .

Semantic typing judgements

$$E; L \mid T_1 \models l \Rightarrow x. T_2 := \\ \forall t. \{ \llbracket E \rrbracket * \llbracket L \rrbracket * [Na : t] * \llbracket T_1 \rrbracket(t) \} \mid \{ v. \llbracket L \rrbracket * [Na : t] * \llbracket T_2 \rrbracket_{[x \leftarrow v]}(t) \}$$

Semantic typing judgment for function bodies is similar.

Adequacy follows from the soundness of the separation logic.

- If we have $\emptyset \mid \emptyset; \emptyset \mid \emptyset \models f \Rightarrow x. x \triangleleft \mathbf{fn}() \rightarrow \Pi[]$, then we have a Hoare triple with a trivial precondition. Thus f is safe.

Proving the semantic typing rules amounts to proving a Hoare triple.

- We can use the rules of the program logic, just like when proving programs.
- Proof of the fundamental theorem: translating a typing derivation to a Hoare logic derivation.

Semantic typing judgements

$E; L \mid T_1 \models I \Rightarrow x. T_2 :=$

$\forall t. \{ \llbracket E \rrbracket * \llbracket L \rrbracket * [Na : t] * \llbracket T_1 \rrbracket(t) \} / \{ v. \llbracket L \rrbracket * [Na : t] * \llbracket T_2 \rrbracket_{[x \leftarrow v]}(t) \}$

Semantic typing

Remaining problem: define semantic interpretation of types
 $\llbracket \tau \rrbracket.\text{own}(t, \bar{v})$.

Adequacy follows

- If we have $\emptyset \mid \emptyset; \emptyset \mid \emptyset \models f \Rightarrow x. x \triangleleft \text{fn}() \rightarrow \Pi \rrbracket$, then we have a Hoare triple with a trivial precondition. Thus f is safe.

Proving the semantic typing rules amounts to proving a Hoare triple.

- We can use the rules of the program logic, just like when proving programs.
- Proof of the fundamental theorem: translating a typing derivation to a Hoare logic derivation.

Basic types

$\llbracket \text{bool} \rrbracket.\text{own}(t, \bar{v}) := ?$

$\llbracket \text{int} \rrbracket.\text{own}(t, \bar{v}) := ?$

Basic types

$$\llbracket \text{bool} \rrbracket.\text{own}(t, \bar{v}) := \bar{v} = \text{true} \vee \bar{v} = \text{false}$$
$$\llbracket \text{int} \rrbracket.\text{own}(t, \bar{v}) := \exists z \in \mathbb{Z}. \bar{v} = z$$

So far, so good...

Products

$$\llbracket \tau_1 \times \tau_2 \rrbracket.\text{own}(t, \bar{v}) := ?$$

Products

$$\llbracket \tau_1 \times \tau_2 \rrbracket.\text{own}(t, \bar{v}) := \exists \bar{v}_1 \bar{v}_2. \bar{v} = \bar{v}_1 \bar{v}_2 * \llbracket \tau_1 \rrbracket.\text{own}(t, \bar{v}_1) * \llbracket \tau_2 \rrbracket.\text{own}(t, \bar{v}_2)$$

Okay, that seems to work...

Fully owned pointer

Recall: **own** τ is λ_{Rust} equivalent of **Box**<T>.

$$\llbracket \text{own } \tau \rrbracket.\text{own}(t, \bar{v}) := ?$$

Fully owned pointer

Recall: **own** τ is λ_{Rust} equivalent of **Box**<T>.

$$\llbracket \mathbf{own} \tau \rrbracket.\mathbf{own}(t, \bar{v}) := \exists \ell \bar{w}. \bar{v} = \ell * \ell \mapsto \bar{w} * \triangleright \llbracket \tau \rrbracket.\mathbf{own}(t, \bar{w})$$

We claim the ownership of memory at ℓ containing $\bar{w}$, but also the ownership related to τ , recursively.

Important remark: $\ell \mapsto \bar{w}$ and $\llbracket \tau \rrbracket.\mathbf{own}(t, \bar{w})$ are **separated**.

If $\mathbf{x}$: **Box**<T>, then nothing in $*\mathbf{x}$ can point back to $\mathbf{x}$.

A form of non-aliasing.

Unique borrows

$$\llbracket \&_{\text{mut}}^{\kappa} \tau \rrbracket.\text{own}(t, \bar{v}) := ?$$

Unique borrows

$$\llbracket \&_{\text{mut}}^{\kappa} \tau \rrbracket.\text{own}(t, \bar{v}) := ?$$

Problem: we do not claim full ownership.

We claim **ownership up to κ** . We can use it only when κ is alive.

The lifetime logic

We **extend the separation logic** with new connectives:

- $\&_{\text{full}}^{\kappa} P$: ownership of P when κ is still alive.
- $[\kappa]$: resource witnessing that κ is still alive.
- $[\dagger\kappa]$: assertion witnessing that κ is dead.
 - **persistent** assertion: not a resource, duplicable.

And prove a few rules (using Iris mechanisms):

$$\text{True} \multimap \exists \kappa. [\kappa] * ([\kappa] \multimap [\dagger\kappa]) \quad \triangleright P \multimap \&_{\text{full}}^{\kappa} P * ([\dagger\kappa] \multimap \triangleright P)$$

$$\&_{\text{full}}^{\kappa} P * [\kappa] \multimap \triangleright P * (\triangleright P \multimap \&_{\text{full}}^{\kappa} P * [\kappa])$$

($\multimap$ and $\multimap$ are kinds of implication in Iris separation logic.)

Modeling unique borrows

$$\llbracket \&_{\text{mut}}^{\kappa} \tau \rrbracket.\text{own}(t, \bar{v}) := ?$$

Introduction

Syntactic type system

λ_{Rust}

Type system

Building the logical relation

Setup

Interpreting simple
types

Lifetime logic

Supporting unsafe code

Modeling unique borrows

$$\llbracket \&_{\text{mut}}^{\kappa} \tau \rrbracket.\text{own}(t, \bar{v}) := \exists \ell. \bar{v} = \ell * \&_{\text{full}}^{\kappa} (\exists \bar{w}. \ell \mapsto w * \llbracket \tau \rrbracket.\text{own}(t, \bar{w}))$$

Introduction

Syntactic type system

λ_{Rust}

Type system

Building the logical relation

Setup

Interpreting simple
types

Lifetime logic

Supporting unsafe code

Modeling unique borrows

$$\llbracket \&_{\text{mut}}^{\kappa} \tau \rrbracket.\text{own}(t, \bar{v}) := \exists \ell. \bar{v} = \ell * \&_{\text{full}}^{\kappa} (\exists \bar{w}. \ell \mapsto w * \llbracket \tau \rrbracket.\text{own}(t, \bar{w}))$$

Creating a borrow:

- We use $\triangleright P \equiv * \&_{\text{full}}^{\kappa} P * ([\dagger^{\kappa}] \equiv * \triangleright P)$.
- We place $[\dagger^{\kappa}] \equiv * \triangleright P$ in the interpretation of the frozen typing context element:

$$\llbracket p \triangleleft^{\dagger^{\kappa}} \tau \rrbracket := [\dagger^{\kappa}] \equiv * \llbracket \tau \rrbracket.\text{own}(t, p)$$

Modeling unique borrows

$$\llbracket \&_{\text{mut}}^{\kappa} \tau \rrbracket.\text{own}(t, \bar{v}) := \exists \ell. \bar{v} = \ell * \&_{\text{full}}^{\kappa} (\exists \bar{w}. \ell \mapsto w * \llbracket \tau \rrbracket.\text{own}(t, \bar{w}))$$

Using a borrow:

- After getting a lifetime token $[\kappa]$, we temporarily “open” $\&_{\text{full}}^{\kappa} P$ using the rule:

$$\&_{\text{full}}^{\kappa} P * [\kappa] \Rightarrow * \triangleright P * (\triangleright P \Rightarrow * \&_{\text{full}}^{\kappa} P * [\kappa])$$

Modeling unique borrows

$$\llbracket \&_{\text{mut}}^{\kappa} \tau \rrbracket.\text{own}(t, \bar{v}) := \exists \ell. \bar{v} = \ell * \&_{\text{full}}^{\kappa} (\exists \bar{w}. \ell \mapsto w * \llbracket \tau \rrbracket.\text{own}(t, \bar{w}))$$

Reborrowing:

- Recall the syntactic rule:

$$\frac{E; L \vdash \kappa' \sqsubseteq \kappa}{E; L \vdash p \triangleleft \&_{\text{mut}}^{\kappa} \tau \stackrel{\text{ctx}}{\Rightarrow} p \triangleleft \&_{\text{mut}}^{\kappa'} \tau, p \triangleleft \dagger^{\kappa'} \&_{\text{mut}}^{\kappa} \tau}$$

We use a lifetime logic rule for reborrowing:

$$\kappa' \sqsubseteq \kappa * \&_{\text{full}}^{\kappa} P \Rightarrow * \&_{\text{full}}^{\kappa'} P * ([\dagger^{\kappa'}] \Rightarrow * \&_{\text{full}}^{\kappa} P)$$

1 Introduction

2 Syntactic type system

- λ_{Rust}
- Type system

3 Building the logical relation

- Setup
- Interpreting simple types
- Lifetime logic

4 Supporting unsafe code

The plan

We have seen how to prove the syntactic type system sound.
How do we extend it to, say, `Cell`?

The plan:

- 1 Define $\llbracket \text{cell}(\tau) \rrbracket$
- 2 Prove the semantic typing judgments on functions on cell (`new`, `from_inner`, `get`, `set`, ...).

Example, for `Cell::new`

Example, for `Cell::new`:

```
impl<T> Cell<T>{  
  fn new(x: T) -> Cell<T> { <code> }  
}
```

Where `<code>` is essentially `return x`.

Unfolding the definitions of the semantic judgment, we should have (roughly):

$$\{\llbracket \tau \rrbracket.\text{own}(x, t)\} \text{<code>} \{v. \llbracket \text{cell}(\tau) \rrbracket.\text{own}(v, t)\}$$

Example, for `Cell::new`

Example, for `Cell::new`:

```
impl<T> Cell<T>{  
  fn new(x: T) -> Cell<T> { <code> }  
}
```

Where `<code>` is essentially `return x`.

Unfolding the definitions of the semantic judgment, we should have (roughly):

$$\{\llbracket \tau \rrbracket.\text{own}(x, t)\} \text{<code>} \{v. \llbracket \text{cell}(\tau) \rrbracket.\text{own}(v, t)\}$$

And conversly for `Cell::into_inner...`

Example, for `Cell::new`

Example, for `Cell::new`:

```
impl<T> Cell<T> {  
  fn new(x: T) -> Self {  
    ...  
  }  
}
```

Where `<code>` is

Unfolding the definition

We can go from $\llbracket \tau \rrbracket.\text{own}(v, t)$ to $\llbracket \text{cell}(\tau) \rrbracket.\text{own}(v, t)$ and back, without changing anything in the memory state...

⇒ We really don't have a choice, we must use:

$$\llbracket \text{cell}(\tau) \rrbracket.\text{own}(v, t) := \llbracket \tau \rrbracket.\text{own}(v, t)$$

But... does that mean that semantically $\text{cell}(\tau)$ and τ are equivalent?
No!

And conversely for `Cell::into_inner`:

Sharing predicates

Even though we want $\llbracket \mathbf{cell}(\tau) \rrbracket.\text{own}(v, t)$ and $\llbracket \tau \rrbracket.\text{own}(v, t)$ to be equivalent, we do **not want** $\llbracket \&_{\text{shr}}^{\kappa} \mathbf{cell}(\tau) \rrbracket.\text{own}(v, t)$ and $\llbracket \&_{\text{shr}}^{\kappa} \tau \rrbracket.\text{own}(v, t)$ to be equivalent

Sharing predicates

Even though we want $\llbracket \mathbf{cell}(\tau) \rrbracket.\text{own}(v, t)$ and $\llbracket \tau \rrbracket.\text{own}(v, t)$ to be equivalent, we do **not want** $\llbracket \&_{\text{shr}}^{\kappa} \mathbf{cell}(\tau) \rrbracket.\text{own}(v, t)$ and $\llbracket \&_{\text{shr}}^{\kappa} \tau \rrbracket.\text{own}(v, t)$ to be equivalent

- The ownership predicate of $\&_{\text{shr}}^{\kappa} \tau$ is not defined from the ownership predicate of τ .
- Each type has its **own definition of** $\llbracket \&_{\text{shr}}^{\kappa} \tau \rrbracket.\text{own}(v, t)$.
 - We call this $\llbracket \tau \rrbracket.\text{shr}(\kappa, t, \ell)$, and define:

$$\llbracket \&_{\text{shr}}^{\kappa} \tau \rrbracket.\text{own}(t, \bar{v}) := \exists \ell. \bar{v} = \ell * \llbracket \tau \rrbracket.\text{shr}(\kappa, t, \ell)$$

- $\llbracket \tau \rrbracket.\text{shr}(\kappa, t, \ell)$ is the **sharing predicate** of τ . It describes the sharing protocol of τ .

Introduction

Syntactic type
system

λ_{Rust}
Type system

Building the logical
relation

Setup
Interpreting simple
types
Lifetime logic

Supporting unsafe
code

Sharing predicates

Even though we want $\llbracket \text{cell}(\tau) \rrbracket.\text{own}(v, t)$ and $\llbracket \tau \rrbracket.\text{own}(v, t)$ to be equivalent, we do **not want** $\llbracket \&_{\text{shr}}^{\kappa} \text{cell}(\tau) \rrbracket.\text{own}(v, t)$ and $\llbracket \&_{\text{shr}}^{\kappa} \tau \rrbracket.\text{own}(v, t)$ to be equivalent

- The ownership predicate of $\&_{\text{shr}}^{\kappa} \tau$ is not defined from the ownership predicate of τ .
- Each type has its **own definition of** $\llbracket \&_{\text{shr}}^{\kappa} \tau \rrbracket.\text{own}(v, t)$.
 - We call this $\llbracket \tau \rrbracket.\text{shr}(\kappa, t, \ell)$, and define:

$$\llbracket \&_{\text{shr}}^{\kappa} \tau \rrbracket.\text{own}(t, \bar{v}) := \exists \ell. \bar{v} = \ell * \llbracket \tau \rrbracket.\text{shr}(\kappa, t, \ell)$$

- $\llbracket \tau \rrbracket.\text{shr}(\kappa, t, \ell)$ is the **sharing predicate** of τ . It describes the sharing protocol of τ .

Recall: **Send** types are those for which $\llbracket \tau \rrbracket.\text{own}(t, \bar{v})$ do not depend on thread t .
How should we semantically interpret **Sync**?

Sharing predicates

Even though we want $\llbracket \text{cell}(\tau) \rrbracket.\text{own}(v, t)$ and $\llbracket \tau \rrbracket.\text{own}(v, t)$ to be equivalent, we do **not want** $\llbracket \&_{\text{shr}}^{\kappa} \text{cell}(\tau) \rrbracket.\text{own}(v, t)$ and $\llbracket \&_{\text{shr}}^{\kappa} \tau \rrbracket.\text{own}(v, t)$ to be equivalent

- The ownership predicate of $\&_{\text{shr}}^{\kappa} \tau$ is not defined from the ownership predicate of τ .
- Each type has its **own definition of** $\llbracket \&_{\text{shr}}^{\kappa} \tau \rrbracket.\text{own}(v, t)$.
 - We call this $\llbracket \tau \rrbracket.\text{shr}(\kappa, t, \ell)$, and define:

$$\llbracket \&_{\text{shr}}^{\kappa} \tau \rrbracket.\text{own}(t, \bar{v}) := \exists \ell. \bar{v} = \ell * \llbracket \tau \rrbracket.\text{shr}(\kappa, t, \ell)$$

- $\llbracket \tau \rrbracket.\text{shr}(\kappa, t, \ell)$ is the **sharing predicate** of τ . It describes the sharing protocol of τ .

Recall: **Send** types are those for which $\llbracket \tau \rrbracket.\text{own}(t, \bar{v})$ do not depend on thread t .
Sync types are those for which $\llbracket \tau \rrbracket.\text{shr}(\kappa, t, \ell)$ do not depend on thread t .

- This perfectly matches $\mathbf{T} : \text{Sync} \Leftrightarrow \&\mathbf{T} : \text{Send}$.

Sharing predicates: persistent

In Rust, the type `&T` is **Copy**. I.e., we can duplicate its values freely.

In our model, this means that $\llbracket \tau \rrbracket.\text{shr}(\kappa, t, \ell)$ needs to be a **duplicable** assertion:

$$\llbracket \tau \rrbracket.\text{shr}(\kappa, t, \ell) \vdash \llbracket \tau \rrbracket.\text{shr}(\kappa, t, \ell) * \llbracket \tau \rrbracket.\text{shr}(\kappa, t, \ell)$$

In Iris terms, we say that $\llbracket \tau \rrbracket.\text{shr}(\kappa, t, \ell)$ should be **persistent**.

Defining sharing predicates

The idea is to define the **sharing protocol** during κ for each of these types.

Two examples:

- $\llbracket \text{int} \rrbracket.\text{shr}(\kappa, t, \ell)$ and $\llbracket \text{bool} \rrbracket.\text{shr}(\kappa, t, \ell)$ only allow read access during κ .
- $\llbracket \text{cell}(\tau) \rrbracket.\text{shr}(\kappa, t, \ell)$ allow reading and writing during κ :
 - but only in thread t (token $[\text{Na} : t]$ required),
 - without extracting ownership.
- ...

Sharing predicate for `int`

Question: what is the definition of $\llbracket \text{int} \rrbracket.\text{shr}(\kappa, t, \ell)$?

It should:

- be persistent;
- allow **read-only** dereferencing ℓ when κ is alive.

Sharing predicate for `int`

Question: what is the definition of $\llbracket \text{int} \rrbracket.\text{shr}(\kappa, t, \ell)$?

It should:

- be persistent
- allow `read-only`

In separation logic, $\ell \mapsto_q v$ (where $q \in \mathbb{Q}$) is the **fractional** ownership of memory pointed to by ℓ .

- $\ell \mapsto v$ is the same as $\ell \mapsto_1 v$
- We can split them in pieces to share the ownership:

$$\ell \mapsto_q v * \ell \mapsto_{q'} v \dashv\vdash \ell \mapsto_{q+q'} v$$

- They can be used for reading, but not for writing

Sharing predicate for `int`

Question: what is the definition of $\llbracket \text{int} \rrbracket.\text{shr}(\kappa, t, \ell)$?

It should:

- be persistent
- allow `read-only`

In separation logic, $\ell \mapsto_q v$ (where $q \in \mathbb{Q}$) is the **fractional** ownership of memory pointed to by ℓ .

- $\ell \mapsto v$ is the same as $\ell \mapsto_1 v$
- We can split them in pieces to share the ownership:

$$\ell \mapsto_q v * \ell \mapsto_{q'} v \dashv\vdash \ell \mapsto_{q+q'} v$$

- They can be used for reading, but not for writing
- Unfortunately, they are **not persistent**!

Sharing predicate for `int`

Question: what is the definition of $\llbracket \text{int} \rrbracket.\text{shr}(\kappa, t, \ell)$?

It should:

- be persistent;
- allow **read-only** dereferencing ℓ when κ is alive.

We introduce **fractured borrows** $\&_{\text{frac}}^{\kappa} \lambda q. \Phi(q)$:

- They allow temporary access to some $\Phi(q)$ when given a lifetime token $[\kappa]$:

$$\&_{\text{frac}}^{\kappa} \lambda q. \Phi(q) * [\kappa] \equiv * \exists q. \triangleright \Phi(q) * (\triangleright \Phi(q) \equiv * [\kappa])$$

- They are persistent.

Sharing predicate for `int`

Question: what is the definition of $\llbracket \text{int} \rrbracket.\text{shr}(\kappa, t, \ell)$?

It should:

- be persistent;
- allow **read-only** dereferencing ℓ when κ is alive.

We introduce **fractured borrows** $\&_{\text{frac}}^{\kappa} \lambda q. \Phi(q)$:

- They allow temporary access to some $\Phi(q)$ when given a lifetime token $[\kappa]$:

$$\&_{\text{frac}}^{\kappa} \lambda q. \Phi(q) * [\kappa] \equiv * \exists q. \triangleright \Phi(q) * (\triangleright \Phi(q) \equiv * [\kappa])$$

- They are persistent.

Then, we define:

$$\llbracket \text{int} \rrbracket.\text{shr}(\kappa, t, \ell) := \exists z \in \mathbb{Z}. \&_{\text{frac}}^{\kappa} \lambda q. \ell \mapsto_q z$$

Sharing predicate for **bool**, $\&_{\text{shr}}^{\kappa} \tau$, ...

The same technique applies for the sharing predicate of many **Copy** types: **bool**, **shr**:

$$\llbracket \mathbf{bool} \rrbracket.\text{shr}(\kappa, t, \ell) := \exists b \in \mathbb{B}. \&_{\text{frac}}^{\kappa} \lambda q. \ell \mapsto_q b$$

$$\llbracket \&_{\text{shr}}^{\kappa'} \tau \rrbracket.\text{shr}(\kappa, t, \ell) := \exists \ell'. \&_{\text{frac}}^{\kappa} \lambda q. \ell \mapsto_q \ell' * \llbracket \tau \rrbracket.\text{shr}(\kappa', t, \ell')$$

Sharing predicate for **cell**(τ)

Question: what is the definition of $\llbracket \mathbf{cell}(\tau) \rrbracket.\text{shr}(\kappa, t, \ell)$?

It should:

- be persistent;
- allow **read and write** dereferencing ℓ when κ is alive, but only in thread t .

Sharing predicate for $\text{cell}(\tau)$

Question: what is the definition of $\llbracket \text{cell}(\tau) \rrbracket.\text{shr}(\kappa, t, \ell)$?

It should:

- be persistent;
- allow **read and write** dereferencing ℓ when κ is alive, but only in thread t .

We introduce **non-atomic persistent borrows** $\&_{\text{na}}^{\kappa/t} P$:

- They allow temporary access to P when given a lifetime token $[\kappa]$ and a thread-local token $[\text{Na} : t]$:

$$\&_{\text{na}}^{\kappa/t} P * [\kappa] * [\text{Na} : t] \equiv * \triangleright P * (\triangleright P \equiv * [\kappa] * [\text{Na} : t])$$

- They are persistent.

Then we define:

$$\llbracket \text{cell}(\tau) \rrbracket.\text{shr}(\kappa, t, \ell) := \&_{\text{na}}^{\kappa/t} (\exists \bar{v}. \ell \mapsto \bar{v} * \llbracket \tau \rrbracket.\text{own}(t, \bar{v}))$$

Conclusion

I hope I gave you a taste of what it is to define a logical relation for a large language!

As you have seen, this is quite technical.

Formalizing in Rocq not only improves trust. It also **helps** to manage the various details.

We can add many extensions to this proof. Examples:

- Add other unsafe libraries (`RefCell`, `Rc`, `Mutex`, `Arc`, ...).
- Support for weak memory model.

Agenda for next weeks

- Next week 2026/02/25: **no course**
- 2026/03/03: deadline for programming project
- 2026/03/04: course here with me
- 2026/03/11: final exam